

Embedded Intelligence at the Edge

Johan Kristiansson | Luleå University of Technology
Lecture 1 – Introduction 31 August 2026



Welcome to D7065E

This course teaches how to design and build **autonomous intelligent systems** that **sense, reason, and act** in physical environments, at the intersection of **edge computing**, **AI agents**, and **cyber-physical systems (CPS)**.

- **The project:** instrument a *simulated smart building* with sensors and actuators, then build software that makes real-time control decisions.
- **Open-ended:** teams choose the control problem, architecture, communication patterns, and autonomous approach. There is no single correct solution.
- **A systems engineering course:** the focus is architecture, communication, data pipelines, testing, and integration, not advanced AI research.

Today

Part A

The course.

How it runs, what you deliver, how it is graded and examined.

Part B

The field.

An introduction to cyber-physical systems, the edge, and autonomy.

Part C

The system.

BuildSim, the architecture, and what you build.

Part A

The course

What you will learn

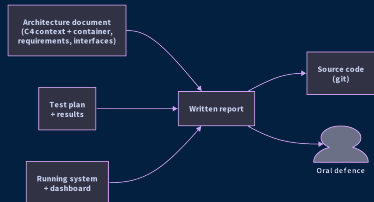
- Design **CPS architectures** connecting sensors, computation, and actuators into autonomous control loops.
- Apply **model-based, specification-driven** methods to design, implement, and validate systems.
- Build **data pipelines** that collect, store, and transform sensor data for decision-making.
- Develop **autonomous services and agents** that observe state, reason, and act.
- Evaluate **resilience, safety, scalability, and performance** under failures and uncertainty.

How the course runs

1. Choose a use case (or propose your own).
2. Write an **architecture document** and design specification.
3. Write a **test plan**.
4. Get the design **approved** at the week-3 checkpoint.
5. Implement the system.
6. Demonstrate and defend your results.

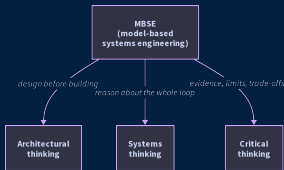
The architecture document is a **living document**: start it once the proposal is approved, and refine it as the system is built.

What you deliver



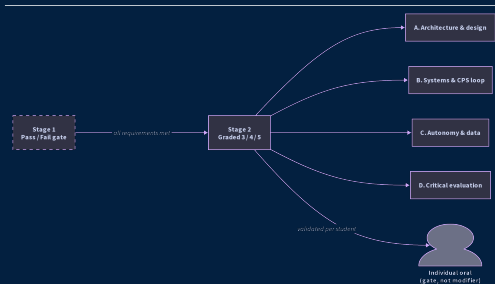
An **architecture document** (C4 context + container, requirements table, interface contracts), a **test plan** with results, the **running system + dashboard**, a **written report**, the **source code** in git, and an **individual oral defence**.

Three kinds of thinking



The course develops **architectural**, **systems**, and **critical** thinking. Your grade reflects these skills, not the number of features or the sophistication of the AI. A simple system that is well-designed, well-justified, and critically evaluated scores higher than a complex one that cannot be explained.

How it is graded



Stage 1 is a Pass/Fail gate of mandatory requirements, miss one and the project is **U** until fixed. **Stage 2** grades four dimensions (A–D) at levels **3/4/5**, judged *holistically*: a serious weakness in one dimension limits the grade.

The four graded dimensions

- **A. Architecture & design** — documented, justified, evaluated (LO2).
- **B. Systems integration & the CPS loop** — the whole loop, failure behaviour, emergent effects (LO1, LO3).
- **C. Autonomous services & data engineering** — decisions backed by a data pipeline, data-quality handling (LO1, LO3).
- **D. Critical evaluation** — testing, fault injection, stress and breaking-point analysis, honest critique (LO4).

Expectations and scope

- This is a **systems engineering** course: architecture, communication, pipelines, testing, and integration come first.
- Apply core **distributed-systems** concepts: decomposition, APIs, messaging, containers, fault handling.
- You are *not* expected to implement consensus protocols, distributed databases, or large-scale cloud infrastructure, those come in later courses.
- **Simple, well-done beats complex and unexplained.** More AI does not mean a higher grade.

Examination

- The project is done **in pairs**; it is open-ended, with no single correct answer.
- Each student takes an **individual oral exam** (~30 min): a 10-minute live demo, then a 20-minute **whiteboard** session, *no slides or notes*.
- The oral is a **gate, not a modifier**: it cannot raise the project grade, only confirm the level each student can *personally* explain and defend.
- Teammates may receive **different grades**. Cannot defend your own system $\Rightarrow$ U.

The oral exam: what you must be able to do

Draw and explain, from memory: the **architecture** and component responsibilities; how the system **senses, reasons, and acts**; how **data flows** from sensing to action; and your **testing** and failure behaviour.

Be ready for questions such as:

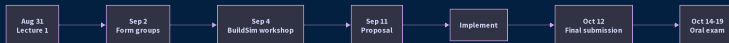
- What makes a system *autonomous* rather than *automated*?
- What happens if a sensor lies, or a decision component becomes unavailable?
- Why this communication pattern? What are the main failure modes? How would it scale?

AI-assisted development

Agentic coding tools (Claude Code, Codex, Cursor, Cline, Aider) can autonomously read code, edit files, run commands, and write tests, and are **strongly encouraged**; they can dramatically increase development speed and project scope.

- You remain **responsible** for every design decision, line of code, test, and conclusion.
- The oral exam assesses **your** understanding, not the AI's ability to generate the system.

Lectures and deadlines



Lecture 1 (today): introduction. **Sep 2**: form lab groups. **Sep 4**: BuildSim workshop. **Sep 11**: submit the proposal. Design approved by week 3, then build. **Oct 12**: final submission. **Oct 14–19**: demonstration and oral examination.

Part B

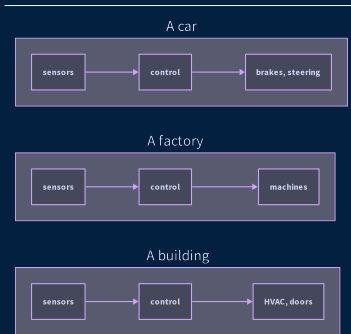
Cyber-physical systems, edge computing, and autonomy

What is a cyber-physical system?



A **cyber-physical system (CPS)** tightly integrates computation, networking, and physical processes [1,2]. It runs a continuous **sense** → **compute** → **act** loop: the *cyber* part senses only through sensors and acts only through actuators; the *physical* part evolves by physics and human behaviour, and never stops, even when no computer is watching.

Cyber-physical systems are everywhere



Described as the *next computing revolution* [2], CPS underpin cars, factories, energy grids, and buildings: **sensors** → **control** → **actuators** in a continuous loop. This course uses a *building* as its cyber-physical system.

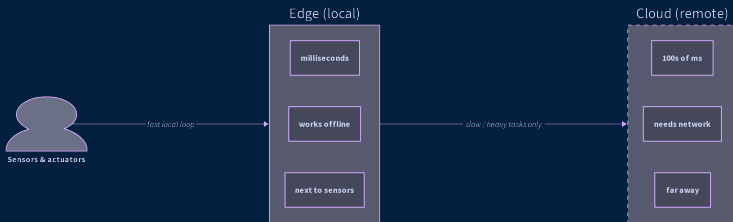
From cloud to edge

As billions of sensors came online, sending every reading to a distant data centre became too slow, too costly in bandwidth, and a single point of failure.

- **Fog computing** [5] introduced a layer between devices and the cloud to support latency-sensitive, geographically distributed applications.
- **Edge computing** [3,4] pushes computation to the edge of the network, *next to the data sources*, rather than into the cloud.

The shift is driven by latency, bandwidth, reliability, and privacy, the recurring themes of *edge intelligence*.

Edge computing: bring computation to the data



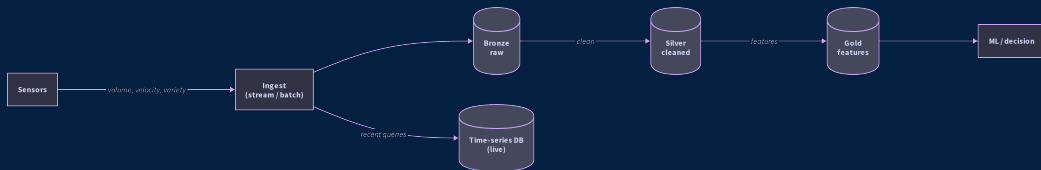
Edge computing places computation next to the sensors: millisecond latency, and it keeps working when the network does not [3,4]. The **cloud** is far away and depends on connectivity. A real-time control loop belongs at the edge.

The edge-cloud continuum



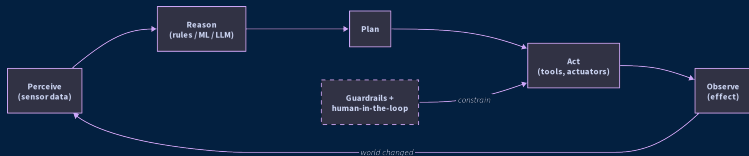
Computation is not edge *or* cloud but a **continuum**: fast inference near the sensors, heavier training pushed out to a local server or the cloud/HPC, with updated models flowing back. Managing resources and placement across this continuum is Learning Outcome 1.

Data engineering for CPS



Sensor data arrives with **volume, velocity, and variety**. It is ingested (stream or batch), refined through a **data lake**, raw → cleaned → feature, the *medallion* pattern, and served from a **time-series database** for live queries, turning raw readings into features an autonomous component can use.

From automation to autonomy



Automation comes in three generations: **rules** → **machine learning** → **agentic AI** [8]. An autonomous agent closes a loop, **perceive** → **reason** → **plan** → **act** → **observe**, using tools to act, with **guardrails** and human-in-the-loop for safety.

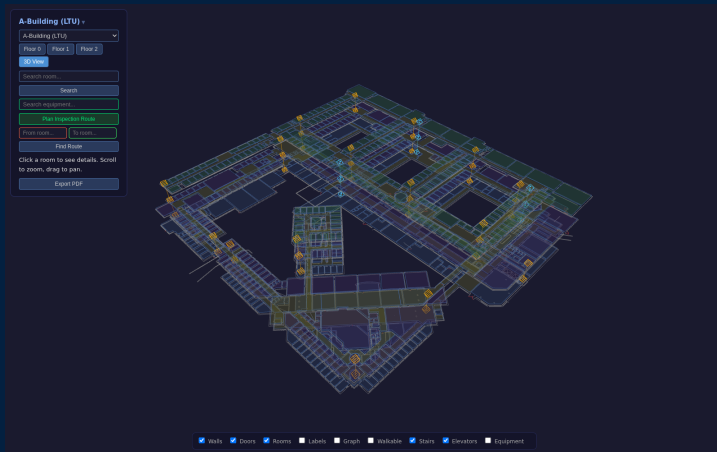
Why the edge matters

- **Latency** — physical processes cannot wait for a round trip to a data centre.
- **Resilience** — the system keeps working when connectivity is lost.
- **Bandwidth** — raw sensor streams are too large to ship everywhere.
- **Privacy** — data can be processed locally instead of leaving the building.
- **Autonomy** — local decisions enable real-time, self-governing behaviour.

Part C

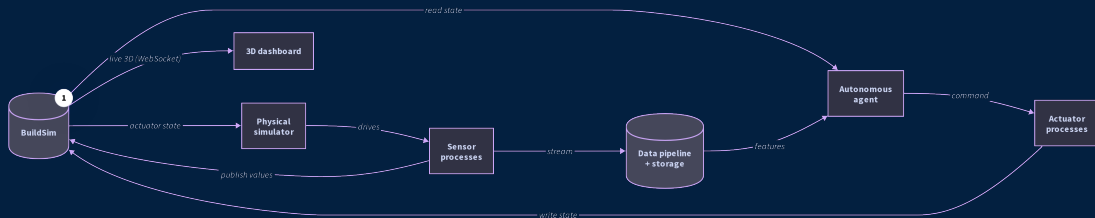
The system you will build

BuildSim: the building, provided for you



BuildSim is a single Go binary that bundles a 3D building model, a REST API, and a live web viewer. It holds

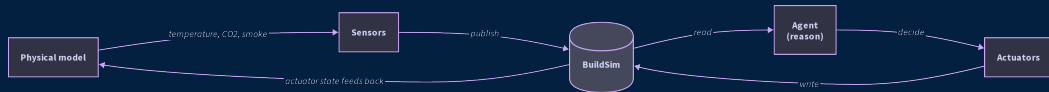
BuildSim is the shared state



1 shared building state

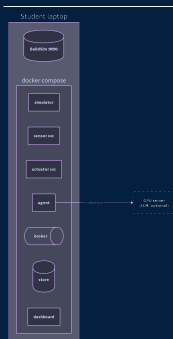
Every component runs as its own process and talks to BuildSim over REST: the **simulator** drives sensor values, the **agent** reads them and commands **actuators**, the **pipeline** stores the stream, and the browser shows the live 3D view.

A closed feedback loop



The simulator turns actuator states into the *next* sensor readings, and the agent turns readings into commands. **Actuator commands must change future sensor readings**; otherwise the loop is open and the control does nothing. This loop is the heart of the project.

Every component is its own process



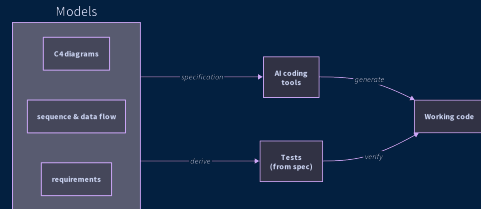
Sensors, actuators, agent, broker, storage, and dashboard each run as a **separate, containerised** service (e.g. `docker compose`) alongside BuildSim. Any process can crash, restart, and re-register. The system must *not* be a single monolith. An optional GPU server can host an LLM.

Why a simulated building?

Testing autonomous control on a real building would be **slow, costly, unsafe, and hard to reproduce**. Instead the project uses **system-level simulation**: the environment, sensors, actuators, communication, and control software are developed and evaluated as one complete CPS.

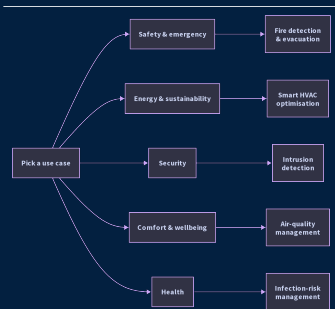
- Model the building *realistically enough* that the architecture would be unchanged on a real building, the goal is not to generate artificial data.
- Test, validate, and iterate under controlled, repeatable conditions.

Specification-first, with MBSE



Model before code: C4 diagrams, sequence and data-flow diagrams, and requirements [6,7]. The models are the **input to AI tools**, and tests derive from the spec. Use **lightweight artefacts**, not SysML, the value is the engineering thinking, not the notation.

Pick a use case



Choose one or **propose your own** (subject to approval). Every project needs sensors, a data pipeline, autonomous decisions, and a dashboard, and must include a **data-driven component** (ML, anomaly detection, forecasting, optimisation), not just fixed thresholds.

What you must build (Pass/Fail requirements)

- Sensor, actuator, and decision components run as **separate, restartable processes**.
- The system works **end-to-end**: sensing → decision → actuation → a visible effect in BuildSim that feeds back into the next readings.
- At least one component performs **autonomous decision-making** from sensor data.
- A **data pipeline** collects, stores, and serves sensor data.
- An **architecture document**: C4 context + container diagrams, requirements table, interfaces.
- A **test plan**, executed: unit tests, ≥ 1 end-to-end test, documented results.
- A **dashboard**, a **written report**, and an oral defence *from memory*.

Missing any one of these means **U (Fail)** until it is addressed.

The physical simulator

Build a simulator that models your use case and produces **realistic sensor data that responds to actuator commands**, closing the feedback loop. Examples:

- **Fire:** smoke rises and spreads to adjacent rooms; sprinklers reduce it.
- **HVAC:** temperature depends on outdoor temp, occupancy (body heat), and HVAC state.
- **Air quality:** CO₂ rises ~40 ppm/person/hour, falls with ventilation.

It need not be physically perfect. Capture trends, correlations, and responses to actuators, then spend your time on **architecture, integration, and testing**.

Occupancy patterns

Occupancy drives temperature, CO₂, lighting, and access, so model it realistically:

- Weekdays: arrive 7–9, lunch 11:30–13:00, leave 16–18; weekends empty.
- Meeting rooms booked in blocks; corridors briefly occupied in transit.

Approaches: physics-based equations, replay of public datasets, a hybrid (real data + injected events), or a generative model. Datasets such as ASHRAE, Building Data Genome, and occupancy-detection sets are available.

Data engineering



Sensor data must flow through a **real pipeline**, not ad-hoc REST calls. Choose and justify a **broker** (MQTT, Kafka, Redis), a **store** (InfluxDB, TimescaleDB, DuckDB, Parquet), and a **feature pipeline**. Integration options: Arrowhead/SOA, ColonyOS for edge-cloud orchestration.

The autonomous component

- At least one component makes **autonomous decisions** from sensor data and drives an actuator.
- **Rule-based, optimisation, ML, or LLM** approaches are all acceptable, if genuine, justified, and integrated. A **data-driven** element is required.
- It can reason about *sensor data* (anomaly detection, prediction), *text* (regulations, manuals), or *both* (an LLM checking readings against rules).

More sophisticated AI does not mean a higher grade. The AI is one part of the system.

Implementation

- Each sensor and actuator is its **own process**, preferably a Docker container, independently startable, stoppable, and restartable.
- **Go** is recommended (single static binaries, strong concurrency, BuildSim is Go). Other languages may be used if strongly motivated.
- Everything runs **locally** on the laptop: `./buildsim start` plus `docker compose up`.

References and further reading

1. E. A. Lee. Cyber Physical Systems: Design Challenges. *IEEE ISORC*, 2008.
2. R. Rajkumar, I. Lee, L. Sha, J. Stankovic. Cyber-Physical Systems: The Next Computing Revolution. *DAC*, 2010.
3. W. Shi, J. Cao, Q. Zhang, Y. Li, L. Xu. Edge Computing: Vision and Challenges. *IEEE Internet of Things J.*, 2016.
4. M. Satyanarayanan. The Emergence of Edge Computing. *Computer*, 2017.
5. F. Bonomi, R. Milito, J. Zhu, S. Addepalli. Fog Computing and Its Role in the Internet of Things. *MCC Workshop*, 2012.
6. J. A. Estefan. Survey of Model-Based Systems Engineering (MBSE) Methodologies. *INCOSE*, 2007.
7. S. Brown. Software Architecture for Developers, Vol. 2, the C4 model. 2018.
8. S. Russell, P. Norvig. Artificial Intelligence: A Modern Approach. 4th ed., 2021.

Syllabus: ltu.se/.../course-syllabus?id=D7065E • C4 model: c4model.com • BuildSim: buildingsim/docs/lab-quickstart.md

